Machine Teaching

Human-Robot Interaction, HW 12

Cole Wendrowski

Wednesday, April 29, 2026

Code Overview

main.py

In short, this file handles the dataset generation. It has a dynamics method that controls how the actions affect the states (movement). The get_action method generates actions from points toward their goal. The state-action plots are created here. We then define the state(s) that we wish to train and test on. These consist of multiple 4x1 arrays containing the robot (x, y) and the goal (x, y). get_dataset then computes each points' action (pointing towards the corresponding goal) and creates the pickle file dataset of state-action pairs. In my design, I have three datasets containing 5, 10, and 20 state-action pairs.

train_policy.py

This file trains the policy on the dataset(s). It follows a standard PyTorch training process, from declaring the model and optimizer, to training for a defined number of epochs (500) with a specified learning rate (0.001). It calculates the loss and updates the weights accordingly through backpropagation. These are stored in three sets of model_weights corresponding with the 5, 10, and 20 state-action pair datasets.

models.py

This file declares a standard multi-layer perceptron neural network using PyTorch.

test_policy.py

This file tests the trained policies. It runs 500 test iterations to ensure maximal accuracy. Each iteration, random points are sampled and the trained policy is applied to control the point towards the goal for 20 steps. The error for each trajectory is calculated. We then plot the trajectories and calculate error statistics, including mean, median, min, max, and standard deviation.

Dataset Strategy

This section will walk through the various strategies for dataset curation tested in this homework.

Manual Dataset Creation

My first goal was to create a dataset with as few examples as possible. To accomplish this, I sought to manually curate a diverse dataset that captured as many viable scenarios as possible within just five examples. These are shown below. ‘x’ is the starting position and ‘o’ is the goal position. The subscript denotes the example. Consider examples 1 through 5.

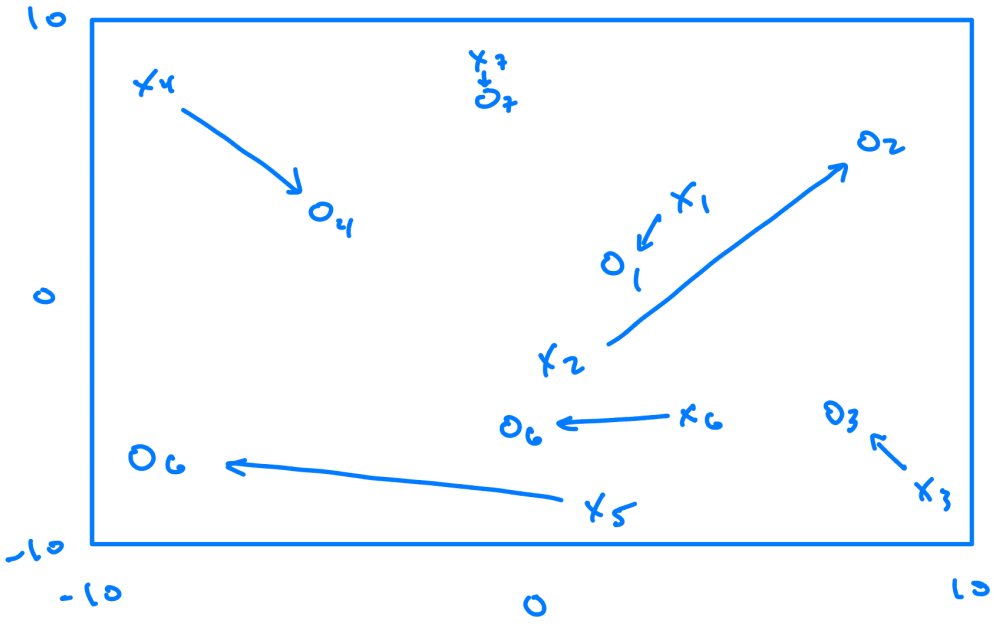

Figure 1 - Manual state-action pair design

These five state-action pairs resulted in relatively impressive error results for so few examples:

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **7.64**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **5.32**

To improve the error, I added two more examples. Example 6 includes a strictly horizontal action and example 7 includes a minutely close adjustment.

This resulted in the following error values:

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **13.47**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **12.04**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **14.28**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **10.4**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **9.1**

(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

here is my average error from the closest goal: **7.55**

As we can observe, the error became worse. This is unlikely to be due to the additional training data and implies that the initial low errors for the 5-example dataset were outliers.

Autoresearch-style Optimization

Inspiration

Next, I wanted to try implementing a recursive self-improvement strategy I read about this week. The inspiration is Andrej Karpathy's autoresearch project. This project uses agentic AI to improve processes through iterative adjustment, training, and testing. When we learned about the homework in class, I realized this would be the perfect workflow to try out this agentic optimization to further improve my datasets.

Task

The task was to create states of 5, 10, and 20 state-action pairs that achieve the lowest possible average error. The agent would test the results for each dataset and iterate to improve them. This enabled the agent to iteratively explore various strategies and adjustments.

The agent could control every stage of the testing process, including:

1. Editing the states in the datasets
2. Training the models on these datasets
3. Testing the models
4. Analyzing the results
5. Determining and implementing improvements

Findings

The agent ran for 11 minutes and 20 seconds. This includes dataset generation, training, and testing time. It discovered that the structure of the game influenced the learning mechanism, and thus the optimal dataset. The policy receives absolute points, rather than relative positions, so the policy needs to learn the relative subtraction operation (goal – robot) from sparse examples. Therefore, the most effective datasets varied spatially and included near-goal offsets. This taught the policy proper wayfinding (direction) and stopping behavior.

Here is the strategy analysis from the agent:

Strategy	Result	Notes
Original/random sparse examples	Poor/inconsistent	Five random states often miss key directions and near-goal behavior.
Long cardinal directions only	Moderate	Better than random for 5 examples. Teaches N/S/E/W movement, but weak on diagonal and stopping behavior.
Long diagonal directions only	Poor	Did not generalize well across the plane or to axis-aligned targets.
Near-goal examples around only the origin	Poor	The model is not translation-invariant, so examples at only [0, 0] do not transfer reliably.
Spatially varied near-goal examples	Strong	Best pattern overall. Teaches the subtraction relation and prevents overshooting.
Mixed random relational search	Best for 20 examples	A searched 20-state set with varied absolute positions and relative offsets gave the lowest error.

I implemented and deployed this project within the class period and thus wanted to minimize testing time (hence why I only ran the agent for 11 minutes). Future work could update this autoresearch process to run indefinitely in 5-minute testing-and-training iterations. This would better mirror the original autoresearch workflow and provide more iterations to improve the dataset and test new strategies.

Datasets

The 5-example dataset consists of each cardinal direction (four examples) + a close offset example. This ensures relatively robust direction determination and stopping behavior. The 10 and 20 example datasets are more complex.

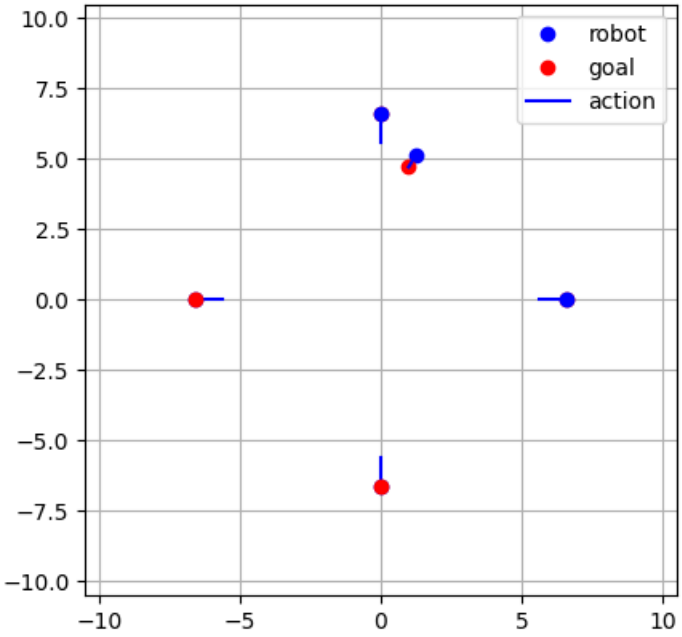

Figure 2 - 5-example dataset

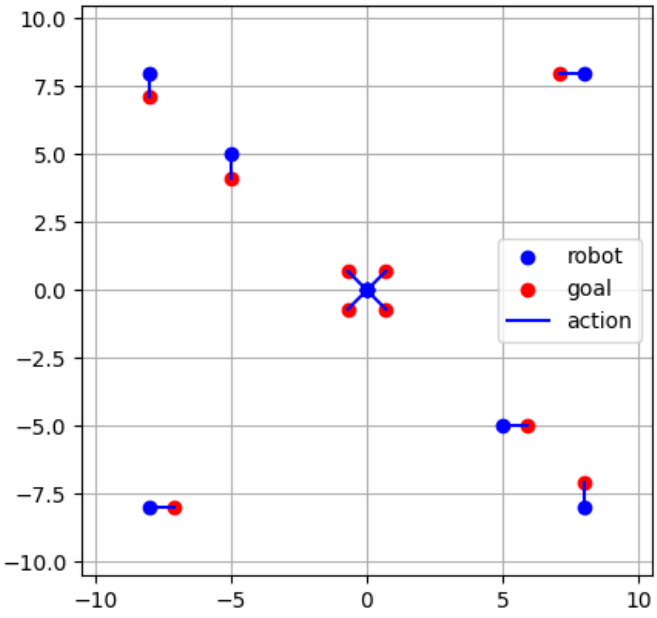

Figure 3 - 10-example dataset

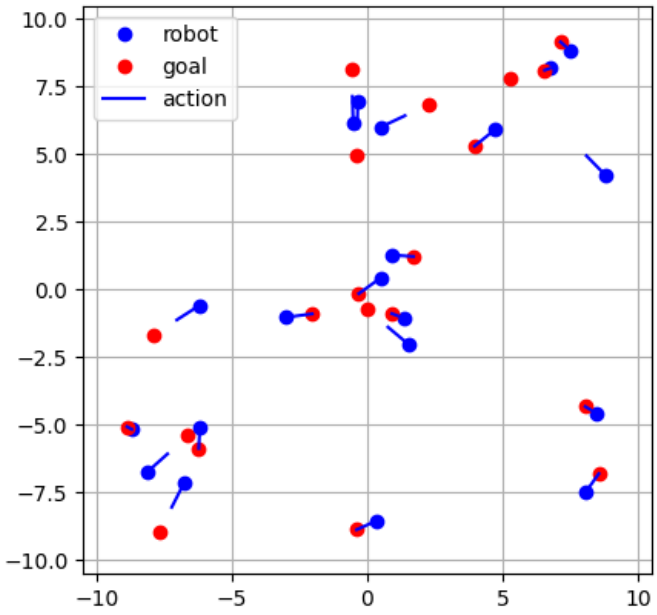

Figure 4 - 20-example dataset

Results

```
(.venv) cole@Coles-MacBook-Pro HRI_HW12 % python3 test_policy.py

Model              Average    Median       Min       Max    Std Dev
-------------------------------------------------------------------
model_weights5       2.716     2.207     0.106    12.412      2.012
model_weights10      1.336     1.230     0.035     4.479      0.841
model_weights20      0.727     0.538     0.014     3.554      0.629
```

We observe a *monumental* improvement. These calculations are from 500 trials per dataset, ensuring the results are robust. The 5-example dataset's average error of 2.716 is better than most classmates' 20 example dataset errors. The 20-example dataset successfully achieves the goal of average error less than one.

General Applications

This project demonstrates two fundamental principles for robot learning and cultivating minimal yet robust datasets.

First, we observe that the model benefits from examples demonstrating archetypal behavior. For example, the 5-example dataset contains the four cardinal directions plus a short offset to teach stopping behavior. From this, the robot can learn the core mechanics of the game. The 10 and 20 example datasets build on this principle, outlining further core movement types and situations.

Second, this project explored agentic optimization. With this approach, the robot can determine and articulate to the human what types of data are most performant. The human can then follow these strategies to optimize their data collection. Additionally, the robot can then progress further and optimize their own data. Using agents, the robot can iteratively test and refine their data to maximize performance.